Modelling for Combinatorial Optimisation (1DL451)
Uppsala University – Autumn 2026
Assignment 3: Scooter Service Problem (SSP)

— Deadline: 13:00 on Friday 11 December 2026 —

The scope of this assignment is Topics 1 to 8: you need ***not*** show any knowledge of subsequent topics. Read the **Report Instructions** and **Grading Rules** at the end of this document ***before*** tackling the following problem(s) upon reading them a first time. It is strongly recommended to prepare and attend the help sessions, as huge time savings may ensue.

The objectives of this assignment are (a) to design a model for a problem with a lot of side constraints, (b) to learn how a complex vehicle routing problem can be modelled with suitable global constraints, (c) to observe experimentally the power of large-neighbourhood search (which is a form of local search), and (d) to see the bigger picture, as an optimisation problem is usually just one aspect of a larger problem, here in the sense that the instance data come from a machine-learning pipeline.

Background

Rented electric scooters are a popular transportation method in urban areas. However, they present logistic challenges for each company that owns a fleet of electric scooters. We tackle here part of the challenge of deciding how to service the scooters, where service employees travel around town with charged batteries in order to replace the batteries of scooters with a too low charge or do some other minor maintenance. We assume each service employee is paid based on the payment values of the actually serviced scooters.

Since a company need not service all the scooters at once, it can decide every now and then which scooters to service, which employee services which scooters, and which route each employee travels. Furthermore, when deciding whether or not to service a particular scooter, the company also wants to take into account other factors than its current charge level, such as the likelihood that someone will want to use it soon, its geographical location, and its need for maintenance. For taking these factors into account, one can use methods of data science in order to create a machine-learning pipeline that, based on historic and current data, generates the matrix of estimated current travel times between any two scooters, as well as predictions (in the form of a scalar per scooter) that represent the service priority and payment value of each scooter.

We consider here what to do once we have such predictions from a machine-learning pipeline: rather than letting the employees decide *individually* for which scooters they grab the intention of service, we use the MiniZinc toolchain in order to make these decisions *automatically* while considering all the employees *collectively*.

The Scooter Service Problem

We are given the following parameters and derived parameters:

- `nScooters` is the number of scooters;
- `nActualEmployees` is the number of actual employees, excluding a dummy employee;
- `dummy` is the dummy employee;

- `nEmployeesAndDummy` is the number of all employees, including the dummy one;
- `nNodes = nScooters + 2 * nEmployeesAndDummy` is the number of nodes in a directed graph, where the nodes in `1..nScooters` represent the current scooter locations, and the nodes in `(nScooters+1)..nNodes` represent the start and end locations of all the employees, as explained in the next item;
- for each employee `e` in `1..nEmployeesAndDummy`:
  - `StartNode[e]` is the start node of `e`, namely `nScooters+e`;
  - `EndNode[e]` is the end node of `e`, namely `nScooters+nEmployeesAndDummy+e`;
- `TravelTime[f,t]` is the estimated time of travelling from node `f` to node `t`, both in `1..nScooters+2`, *plus* servicing the scooter at `t`, if any; all employees start and end at the same nodes, namely `nScooters+1` and `nScooters+2`, respectively; note that `TravelTime` *cannot* be assumed to be symmetric;
- `TT` is the extension of matrix `TravelTime` (as that matrix assumes that there is only the dummy employee) in order to accommodate the actual employees;
- for each scooter `s` in `1..nScooters`:
  - `Priority[s]` is the priority of some actual employee servicing `s`: a higher value means a higher priority;
  - `Payment[s]` is the payment that any actual employee would receive for servicing `s`;
- `minNumScooters` is the minimum number of scooters each actual employee can service;
- `maxNumScooters` is the maximum number of scooters each actual employee can service;
- `maxTime` is the preferred maximum total travel time of each actual employee.

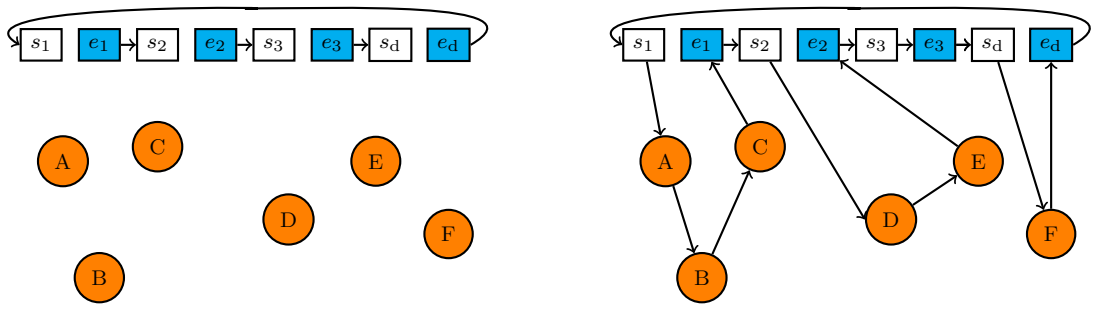
The *scooter service problem* (*SSP*) is to decide, within the complete directed graph induced by the nodes described above, the route of each employee, which then also determines the scooters that each actual employee services. The constraints are as follows:

- Each scooter is either serviced by at most one actual employee or simply visited by the dummy employee without servicing it.
- Each actual employee services either `0` scooters (and has an empty route) or from `minNumScooters` to `maxNumScooters` scooters.

There are three simultaneous objectives:

- Minimise `totalOvertime`, which is the total overtime of all the actual employees, where the overtime of an employee is the amount by which their total travel time exceeds `maxTime`.
- Maximise `totalPayment`, which is the total payment that all the actual employees receive. Note that this makes sense even from the company point of view: the scooters that have a higher payment value are normally prioritised.
- Minimise `totalPriorityLoss`, which is the total priority of all the scooters that are simply visited by the dummy employee without servicing it.

In order to combine these objectives into a single objective, we minimise a weighted sum in order to balance them:



(a) Constraining the successor of each end node (b) A single circuit under the giant-tour formulation

Figure 1: The giant-tour viewpoint, for three actual employees and six scooters

```
minimize alpha * totalOvertime
       - beta  * totalPayment
       + gamma * totalPriorityLoss
```

where `alpha`, `beta`, and `gamma` are additional parameters. Note that we do not minimise the total travel time of all the actual employees: this can be done as a post-processing step where we optimise the route of each actual employee, but we do not consider doing so in this assignment.

Viewpoint

In order to model the SSP, we use a so-called *giant-tour formulation* to represent the routing of all the employees, where all their routes are merged into a single circuit. We more or less transform our vehicle routing problem (VRP) into a travelling salesperson problem (TSP). Formally, we use an array `Succ` of so-called *successor variables*, indexed by all the nodes (the scooters, as well as all the employee start and end nodes), where `Succ[n]` denotes the node that is visited after node `n`, and we state a single `circuit` constraint on these variables. For this formulation to be correct, you must state constraints that force the successor of each employee's end node to be the start node of the circularly next employee.

Figure 1 is a sketch of this formulation for three actual employees and six scooters: the white and blue squares are respectively the start and end nodes of an employee, while the orange circles are the scooter nodes. Subfigure 1a shows the additional constraints described above, enforcing that the successor of each blue end node is the white start node of the circularly next employee. Subfigure 1b shows a solution, where actual employee 1 services scooters A, B, and C, in this order; actual employee 2 services scooter D and then E; actual employee 3 services no scooters; and the dummy employee simply visits scooter F without servicing it.

A skeleton MiniZinc model and instances of varying sizes and difficulty, in the form of datafiles using the parameter names above, are available. For brevity, you need not import lines 1 to 68 of the skeleton model `SSP.mzn`, if you do not change them, into your report.

Tasks

Perform the following sequence of tasks:[1]

A. Write and evaluate a MiniZinc model in files called `SSP.mzn` and `SSP-solve.mzn`, where the latter imports the former and adds the `solve` statement.

   Hint: We provide most of the decision variables needed to model this problem, but you can remove some if you find no use for them, and you might need additional ones.

[1]Solo teams, except PhD students, may skip Task B, but are highly encouraged to do it nevertheless.

Warning: It is very rare but you may run into what is called *numerical instability* (due to integers being relaxed into floats) when a MIP-based backend claims that a feasible sub-optimal solution (as per the other backends) is optimal. Sometimes, very small changes to the model remove the bug, but do not spend much time on this and just mention the instability in reply to the question "Are there any contradictions between the results?" of the report.

B. Write a large-neighbourhood search (LNS) strategy in a file called `SSP-lns.mzn`, which also imports the `SSP.mzn` model, by adding the annotations `restart_constant(`ρ`)` and `relax_and_reconstruct(Succ,`σ`)` to the `solve` statement (just *before* the `minimize` keyword), for Gecode to perform LNS for solving the SSP.

   Determine experimentally (but without reporting how) good values for ρ (denoting the constant number of search nodes after which the solver restarts) and σ (denoting the probability, as an integer percentage, of each variable of the array `Succ` being fixed upon a restart to its value in the previous solution): try *a few* pairs $\langle \rho, \sigma \rangle$ in $\{3000, 4000, \ldots, 10000\} \times \{60, 70, 80, 90\}$, on the *largest* instances; there might be better pairs for your model, but we only consider those. Evaluate `SSP-lns.mzn` on *all* instances, but only under Gecode: when invoking the script `run_backends.sh` use the flag `--backends gecode`.

   With LNS, Gecode no longer performs systematic search, but *local search*, and is thus no longer able to prove optimality unless the objective value reaches the lower bound, but it can often find solutions of much higher quality before timing out. Based on your evaluation, is this a reasonable trade-off here? Why?

For the evaluation(s), use all the provided instances. ***Use a timeout of 5 CPU minutes per instance in order to avoid too long solving times***. For your convenience, here are the minimal objective values for some small instances (in real life, you do not know any of the optima when you start modelling a problem):

instance name	objective value
SSP-005_003_01	−266
SSP-009_006_03	42
SSP-010_008_01	−636
SSP-012_006_03	−538
SSP-023_017_03	−1239

Report Instructions

1. You ***must*** fill in the provided skeleton report and skeleton model(s) and produce a report of maximum 10 pages. You ***must*** inspect your model(s) using our checklist. See the demo report for generic instructions and the expected quality of each part of the provided skeleton report. The specific writing instructions are in comments that start with '`%%`' in the LaTeX source of the skeleton report and should be followed even when not using LaTeX. All running text should be black: before submitting, comment away line 15 (which typesets the placeholders in blue) and uncomment line 16 (which typesets them in black). In the provided MiniZinc skeleton model(s), the placeholders are '`...`' and the main model items are delimited by comments that start with '`%%`'.

2. Spellcheck the report and the comments in ***all*** models, in order to show both self-respect and respect for your readers. Thoroughly proofread the report, at least once per teammate, and ideally grammar-check it. In case you are curious about technical writing: see the English Style Guide of UU, the technical-writing Style Manual of the Optimisation research

group, a list of common errors in English usage, and a list of common errors in English usage by native Swedish speakers.

3. Remember that when submitting you implicitly certify (a) that your report and all its uploaded attachments were produced solely by your team, except where explicitly stated otherwise and clearly referenced, (b) that each teammate can individually explain any part at the mandatory grading session, and (c) that your report and attachments are not freely accessible on a public repository.

4. Submit (by only ***one*** of the teammates) by the given ***hard*** deadline two files via *Studium*: your report as a ***PDF*** file (all other formats will be rejected) and a compressed folder in ***ZIP*** format with all source-code files (including all MiniZinc models, plus possibly a software pipeline for the project) and datafiles (only for the project).

Grading Rules (stricter than the ones for Assignment 1)

If *all* tasks have been tackled within 10 pages, ***and*** *all* requested models are uploaded in files with the imposed names, structure, comments (in the model), and explanations (in the report) exemplified in the demo report, ***and*** *all* models produce *correct* solutions under *infinite* timeout to *all* instances under backends of *all* considered solving technologies that perform systematic search, ***and*** *all* models produce (*near-*)*optimal* solutions in *reasonable* time to *most* instances under backends of *at least two* solving technologies (under MiniZinc version 2.10.1 on a Linux computer of the IT department), ***then*** your *initial* score is $i \in 1\,..\,5$ points (read on), ***else*** your *final* score is 0 points and you *fail* this assignment. Furthermore:

- ***If*** $i \in 3\,..\,5$ points (meaning that *all* models have *good enough* comments and explanations ***and*** *all* task answers are *correct enough*), ***then*** your *final* score is either $i-1$ or i (or no score), depending on your individual performance at the mandatory grading session.

- ***If*** $i \in 1\,..\,2$ points (meaning that *some* models have *poor* comments or explanations ***or*** *some* task answers have *severe errors*), ***then*** your *final* score is either $i-1$ or i or $i+1$ (or no score), depending on (a) your individual ability to correct this, if asked to do so, and (b) your individual performance, both during the mandatory grading session.

Also, ***if*** an assistant figures out a minor fix that is needed to make some model run as per our instructions above, ***then***, instead of giving a final score of 0 points, the assistant may at their discretion deduct 1 point from the initial score earned upon the fix.

Re-examination will be conducted during the general re-examination period from 16 to 28 August 2027: contact the head teacher by email minimum 2 weeks in advance, so that the questions can be set on time.